

채팅 로그 — 2026-04-23

프로젝트: tesla-open-can-mod
주요 주제: 실차 테스트 준비, 진단 방법, 아키텍처 이해, 스킬 생성, BLE 모니터링 가능성 검토, BUS-OFF 재검증, 실차 디버그 로그 추가

대화 흐름 요약

1. 스킬 생성 요청

질문: "현재 지금까지 나와 대화했던 내용을 바탕으로 스킬을 만들어줄래?"

생성된 스킬:

`.github/skills/tesla-can-twai-diagnosis/SKILL.md`

포함된 7단계 워크플로:

단계	내용
1. 증상 수집	<code>busoff_count</code> , <code>tx_err_peak</code> , <code>frameHz</code> 등 정상/이상 기준값 표
2. git 히스토리 추적	<code>twai_initiate_recovery</code> , COOLDOWN 단축, <code>WIFI_AP_STA</code> 위험 패턴 식별 커맨드
3. 오리지널 비교	ev-open-can-tools 기준값 비교표
4. 하드웨어 격리	접지 진단, OBD-II 핀 역할
5. 최소 수정 원칙	필수/권장/선택 3단계 분류
6. 검증 기준	차량 캡처 체크리스트 + 성공 수치 기준
7. 커밋 규칙	롤백 메시지 포함 커밋 포맷

자주 발생하는 실수 5가지 (오늘 분석에서 실제 나온 것들):

- GND 연결 안 하고 소프트웨어만 수정
- COOLDOWN만 줄이고 복구 방식 유지
- `busoff_count` 카운터 추가 후 "갑자기 많이 발생" 착각
- WiFi AP+STA 유지하면서 ESP-NOW 사용
- `twai_initiate_recovery()` "공식 API니까 맞다" 오해

copilot-instructions.md 에 스킬 등록 완료:

```
### Skills (Read SKILL.md before working on related tasks)
- `TWAI B채널 BUS-OFF 심층 진단`: `.github/skills/tesla-can-twai-diagnosis/SKILL.md`
```

2. 아이폰 BLE 시리얼 모니터링 가능성 검토

질문: "아이폰으로 BLE를 통한 시리얼 출력을 받고 싶는데 수정 많이 필요할까?"

결론: 수정 범위 작음

핵심 이유: logRing 이 이미 존재하고 모든 로그가 거기에 저장되기 때문.

필요한 변경:

파일	변경	규모
platformio.ini	NimBLE 라이브러리 1줄 추가	극소
include/ble_monitor.h	BLE UART 서비스 (새 파일)	~100줄
src/main.cpp	bleMonitorInit() 호출 2~3줄	극소
나머지 파일	변경 없음	0

메모리 영향 예상:

항목	현재	BLE 추가 후
RAM	39.5% (129KB)	~55% (180KB)
Flash	43.5% (855KB)	~55% (1080KB)

구조:

```
logRing.push("...", millis()) ← 기존 코드 그대로
    ↓
[BLE 태스크, 100ms 폴링] ← 새로 추가
    ↓
NUS TX Characteristic Notify ← iPhone으로 전송
```

iPhone에서 nRF Connect 또는 LightBlue (둘 다 무료)로 바로 수신 가능.

주의사항: WiFi + BLE 동시 동작은 ESP32-S3 공식 지원. TWAI BUS-OFF에는 영향 없음. ESP-NOW 간헐적 패킷 지연 가능성은 낮게 존재.

3. 실차 테스트 시 B채널 재부팅 현상 원인 분석

질문: "실제 차량 실험 시 B채널 재부팅 현상이 계속 발생된다면 어디서 문제가 있겠는가?"

먼저 재부팅 종류 구분:

현상	의미	심각도
B채널 Hz가 순간 0 → 복구	<code>recoverWithCooldown()</code> 정상 동작	🟡 정상
웹 대시보드 5초 이상 오프라인 후 재연결	ESP32 전체 재부팅	🔴 심각
시리얼에 "🟢 [B-CH] Task 시작" 재출력	nagKillerTask 또는 전체 재부팅	🔴 심각

원인 우선순위:

순위	원인	확인 방법
🔴 1	차량 전압 강하 (브라운아웃)	시리얼 첫 줄 " <code>Brownout detector</code> "
🔴 2	GND 미접지 → BUS-OFF 연속 → <code>twai_driver_install()</code> 상태 오염	웹 <code>recovery_fail_count > 0</code>
🟡 3	<code>send()</code> 2ms mutex 점유 중 웹서버 충돌 → WDT	$100\text{Hz} \times 2\text{ms} = 20\%$ 점유
🟡 4	<code>nagKillerTask</code> 5초 상태 블록 스택 과다 (<code>char[220]</code> + <code>char[180]</code>)	BLE 추가 시 위험
🟡 5	WiFi + TWAI 2.4GHz 간섭 (간헐적)	TX_ERR 카운터 급등 시

재부팅 리셋 원인 코드:

<code>rst:0x1 (POWERON_RESET)</code>	→ 정상 첫 부팅
<code>rst:0x10 (RTCWDT)</code>	→ 워치독 → 코드 무한루프 의심
<code>rst:0x0c (SW_RESET)</code>	→ OTA 또는 <code>esp_restart()</code> 호출
<code>Brownout detector</code>	→ 전압 강하 → 하드웨어 문제

가장 빠른 진단법:

차량 가기 전 `STAT` 명령 기준값 기록 → 재부팅 후 즉시 `STAT` 재입력 → 카운터가 0으로 리셋됐으면 = 전체 ESP32 재부팅, 유지됐으면 = 드라이버 재초기화.

4. A채널이 안정적인 이유 + B채널 100Hz인 이유

질문: "왜 A채널은 문제가 없는 걸까? 그리고 A채널은 낮은 프레임 속도이고 B채널은 왜 이렇게 빠른가?"

A채널이 안정적인 2가지 이유:

① 하드웨어 격리 (핵심)

A채널: MCP2515 (외부 SPI 칩)

- ESP32와 물리적으로 분리된 CAN 컨트롤러
- WiFi 인터럽트의 영향을 받지 않음
- 자체 TX/RX 버퍼에서 CAN 타이밍 독립 처리

B채널: TWAI (ESP32-S3 내장)

- WiFi 인터럽트와 ESP32 코어 공유
- WiFi 채널 전환 시 TWAI ISR 지연 → ACK 윈도우($\sim 1.5\mu s$) 놓침

② TX를 거의 안 함

A채널: ID 1021 수신 → mux==1 조건부 → 매우 드문 TX

B채널: ID 880 수신마다 에코 = 100Hz TX

- BUS-OFF는 TX 에러 카운터(TEC)가 256 도달 시만 발생
- TX 안 하면 BUS-OFF 원리상 불가능

B채널이 100Hz인 이유:

- ID 880 = Tesla EPAS (전동 파워스티어링) 안전 임계 제어 메시지
- Tesla가 10ms(100Hz)마다 핸즈온 여부 판단 → 설계 상 변경 불가
- NagHandler가 880에 1:1로 에코 → 우리도 100Hz TX

채널 성격 비교:

	A채널 (MCP2515)	B채널 (TWAI)
역할	UI 설정값 수정	실시간 스티어링 제어
Tesla 전송 주기	$\sim 6\text{Hz}$ (저속 설정)	100Hz (안전 임계)
우리 TX	드뭄 (조건부)	100Hz 에코 (항상)
하드웨어	외부 SPI 칩 (격리)	ESP32 내장 (공유)
WiFi 간섭	없음	있음
BUS-OFF 위험	낮음	높음

결론: A채널이 안정적인 건 Tesla가 A채널 메시지를 저속으로 설계했고 별도 하드웨어 칩을 사용하기 때문. B채널 문제는 Tesla의 100Hz 요구사항 + ESP32 내장 컨트롤러의 WiFi 인터럽트 공유 구조가 충돌하는 **구조적 문제**.

5. Frame Rate 계산 방식 상세 설명

질문: "Frame Rate 계산은 어떻게 하는지 자세히 알려줘."

공식:

\1

A채널 계산 위치: `include/app.h` → `appLoop()`

```
loop() 호출 (Core 1, ~1ms 주기)
└ while (appDriver->read(frame)) → _aHzFrames++
└ if (elapsed >= 1000ms)
    frameHz = _aHzFrames * 1000 / elapsed
    _aHzFrames = 0 / _aLastHzMs = now
```

`_aHzFrames`, `_aLastHzMs` 모두 **static 지역변수**로 루프 간 유지.

B채널 계산 위치: `src/main.cpp` → `nagKillerTask()`

```
nagKillerTask (Core 0, 1ms vTaskDelay 주기)
└ while(driverB->read(frame)) → processBChannelFrame() → bHzFrames++
└ if (elapsed >= 1000ms)
    bChannelDiag.frameHz = bHzFrames * 1000 / elapsed
    bHzFrames = 0 / lastHzMs = nowHz
```

오차 발생 요인 4가지:

요인	내용	보정 여부
FreeRTOS 스케줄러 지연	elapsed가 정확히 1000ms 아닐 수 있음	✅ elapsed 분모 사용으로 보정
BUS-OFF 복구 중 드롭	재설치 ~5ms 동안 read() 실패	❌ Hz 순간 낮게 표시
txBackoff 중 RX는 지속	frameHz는 RX 기준, TX 상태 무관	—
TEST 시뮬 중 합산	시뮬 프레임도 <code>bHzFrames++</code>	❌ 실제보다 높게 표시

오늘 적용된 코드 변경

오늘 전반은 **분석/이해/계획 세션**이었고, 후반 세션에서 실차 원인 분리용 **진단 로그만** 추가했다.
기존 제어 로직, 필터, 복구 정책, TX 동작 조건은 변경하지 않았다.

파일	변경 내용	목적
src/main.cpp	[B00T] reset=... 부팅 원인 로그 추가	브라운아웃/WDT/패닉과 BUS-OFF 구분
src/main.cpp	B-TXERR 정상/경고/위험구간 로그 추가	TX 에러 상승 추세 조기 감지
src/main.cpp	B-TWAI 상태변경, B-TX 백오프 드롭 증가, B-RX 루프 포화 로그 추가	실차에서 원인 분기 단축
include/drivers/twai_driver.h	tx_fail, bus_off detected, recover done, periodic reinstall 로그 추가	드라이버 내부 복구 단계 관찰

빌드 결과: SUCCESS (RAM 39.5%, Flash 43.6%)

6. BUS-OFF 원인 재검증 (현재 코드 기준)

질문: "모든 면에서 버스오프가 발생하는 원인 예상. 실차에서 어떤 반응이 나오면 어디를 수정해야 할지 상세하게 다시 검증"

핵심 재검증 결론:

1. 첫 번째 BUS-OFF를 만드는 원인과
2. BUS-OFF가 자주 보이게 만드는 원인은 다르다.

첫 BUS-OFF의 1순위 후보:

- 공통 GND 불량
- 차량 전원 강하 / 브라운아웃
- B채널 송신 path(100Hz echo)의 물리 계층 민감성

BUS-OFF가 많이 보이게 만드는 증폭기:

- 과거 twai_initiate_recovery()
- 과거 BUSOFF_COOLDOWN_MS = 200ms
- Core 0 자원 경쟁 (nag task + DNS + HTTP + ESP-NOW)

현재 코드에서 이미 완화된 것:

- WiFi.mode(WIFI_AP_STA) → WiFi.mode(WIFI_AP)
- twai_initiate_recovery() 제거
- BUSOFF_COOLDOWN_MS = 1000ms 복원

즉, 지금 실차에서 BUS-OFF가 다시 발생하면 복구 정책 자체보다 첫 번째 원인(접지/전원/송신 물리 계층)을 더 우선해서 봐야 한다는 결론으로 재정리했다.

7. 실차 반응 → 원인 → 우선 조치 표

실차 반응	1차 판단	가장 가능성 높은 원인	먼저 볼 곳
B-RX는 ~100Hz 유지, TXerr 만 상승	수신은 정상, 송신만 불안정	공통 GND, CAN 배선, 전원 기준	OBD GND / 보드 GND / CAN H-L
RXerr 와 TXerr 가 같이 상승	물리 계층 흔들림	접지 불량, 노이즈, 전원 기준 불일치	OBD 핀 4/5, GND 연속성
nag OFF 면 안정, nag ON 이면 불안정	B채널 TX path 문제로 범위 확정	echo 송신 + 물리 송신 환경	NagHandler , TWAIDriver::send()
웹 UI 열었을 때만 더 심해짐	Core 0 경쟁이 증상 증폭	HTTP/DNS/ESP-NOW와 nag task 경쟁	Core 0 태스크 배치
recover_success 는 늘지만 busoff_count 도 계속 증가	복구는 되지만 첫 원인은 남아 있음	접지/전원/송신 path	하드웨어 우선
recovery_fail_count > 0	재설치 실패 또는 리셋 계열	전원 강하, 드라이버 상태 오염	reset reason, recover 로그
카운터 전체가 0으로 초기화	드라이버 복구가 아니라 보드 재부팅	Brownout, WDT, Panic	부팅 직후 reset reason

8. 추가한 진단 로그와 해석 기준

추가한 로그 문구:

로그	의미	정상 패턴	불량 패턴
[BOOT] reset=...	부팅 원인	POWERON , SW	BROWNOUT , WDT , PANIC
[DIAG] B-TXERR 경고구간 진입	TX 에러 상승 시작	잠깐 뜨고 복귀	반복되며 위험구간 이동
[DIAG] B-TXERR 위험구간 진입	BUS-OFF 직전 위험	잠깐 후 복귀	이어서 BUS_OFF 발생
[DIAG] B-TWAI 상태변경	RUNNING/BUS_OFF/INIT 전이	RUNNING 유지	BUS_OFF 반복, connected=0
[DIAG] B-TX 백오프 드롭 증가	송신 실패를 백오프로 흡수 중	잠깐 증가 후 안정	지속 증가
[DIAG] B-RX 루프 포화	Core 0 부하 또는 RX burst	드롭	반복 시 스케줄링 문제 의심
[TWAI-DRV] tx_fail ...	드라이버 레벨 송신 실패	거의 없음	자주 발생
[TWAI-DRV] recover done ok=...	복구 결과	ok=1	ok=0 반복

해석 원칙:

- TXERR 위험구간 → BUS_OFF → recover done ok=1 반복 = **복구는 되지만 첫 원인은 미해결**
- BOOT reset=BROWNOUT = **BUS-OFF보다 전원 문제가 우선**
- B-RX 루프 포화 가 웹 사용 시 반복 = **Core 0 경쟁 증폭**
- nag ON 일 때만 tx_fail 증가 = **B채널 TX path 문제 확정에 가까움**

9. 후반 세션 검증 결과

실행한 작업:

- `main.cpp` 에 부팅/상태 전이/에러 구간/루프 포화 진단 로그 추가
- `twai_driver.h` 에 `tx_fail`, `recover start/done`, `periodic reinstall` 로그 추가
- 제어 로직/필터/복구 정책은 유지

빌드 검증:

```
pio run -e lilygo_t2can
```

결과:

- Build: SUCCESS
 - RAM: 39.5% (129452 / 327680 bytes)
 - Flash: 43.6% (857181 / 1966080 bytes)
-

내일 할 일 (차량 테스트 전 체크리스트)

펌웨어 업로드 전

- [] `pio run -e lilygo_t2can -t upload` 로 최신 펌웨어 업로드
- [] 시리얼 모니터로   [B-CH] TWAI 드라이버 초기화 성공 확인

차량 연결 전

- [] OBD-II 핀 4 또는 5 (GND) → T2-CAN 보드 GND 배선 (필수)
- [] `STAT` 명령으로 기준값 기록 (`busoff=0`, `recovery=0`)

차량 연결 후

- [] 시동 직후 (0분) 웹 대시보드 스크린샷
- [] 5~10분 후 스크린샷 (`busoff_count`, `tx_err_peak` 확인)
- [] 재부팅 발생 시 시리얼 첫 줄 리셋 원인 코드 캡처

재부팅 발생 시 진단 순서

1. 시리얼 첫 줄 → `Brownout` 여부 확인
2. `STAT` 카운터 0 리셋 여부 → 전체 재부팅 vs 드라이버 재초기화 구분
3. 웹 `recovery_fail_count > 0` → `twai_driver_install` 실패 여부